Set up for MicroROS on Pico

Phase 1: Linux PC/Raspberry Pi Setup (Building the Agent)

The goal of this phase is to build the micro-ROS Agent (the serial-to-ROS bridge) and all necessary tools on your Linux PC / Embedded device

1. Create Workspace	`mkdir microros_agent_ws && cd microros_agent_ws`	Creates a new, clean workspace for the Agent code.

2. Clone Setup Tools	`mkdir src && cd src`
	
	`git clone -b jazzy https://github.com/micro-ROS/micro_ros_setup.git`
		Navigates into the source directory.
		
		Clones the official setup repository (using the `jazzy` branch).

3. Install Dependencies	`cd ../`
	
	`rosdep update && rosdep install --from-paths src --ignore-src -y`
		Go back to agent root
		
		Installs all required system dependencies (like C++ build tools) for the Agent.

4. Build Setup Package	`colcon build`	Compiles the micro-ROS setup packages themselves.
5. Source Workspace	`source install/setup.bash`	Loads the new micro-ROS commands (like `create_agent_ws.sh`) into your terminal session.
6. Create Agent Source	`ros2 run micro_ros_setup create_agent_ws.sh`	Clones the actual micro-ROS Agent source code into the workspace (`src/uros/`).
7. Build the Agent	`ros2 run micro_ros_setup build_agent.sh`	Compiles the micro-ROS Agent application into a runnable ROS 2 executable. Export to Sheets

Phase 2: Pico Firmware Setup (C++ Client)

The goal is to clone the official micro-ROS Pico example and prepare the build environment for cross-compilation.

1. Install Pico Prerequisites	`sudo apt install build-essential cmake gcc-arm-none-eabi libnewlib-arm-none-eabi doxygen git python3`	Can do this from any directory

2. Make workspace and fetch sources	`mkdir -p ~/pico/src`
	
	`cd ~/pico/src`
	
	`git clone --recurse-submodules`
		Create a unique workspace to lay the pico SDKs into

	`https://github.com/raspberrypi/pico-sdk.git`
	
	`git clone -b jazzy https://github.com/micro-ROS/micro_ros_raspberrypi_pico_sdk`
		The first repository is the Pi Pico SDK provided by the Pi foundation.
		
		The second contains a precompiled micro-ROS stack together with a hello-world-like example.

3. Set export path for pico sdk	`export PICO_SDK_PATH=~/pico/src/pico-sdk`	

4. Build and compile the code to get the example .uf2 file for flashing to Pico	`cd pico/src/micro_ros_raspberrypi_pico_sdk`
	
	`mkdir build`
	
	`cd build`
	
	`cmake ..`
	
	`make`
		Navigate to the micro-ROS client source
		
		Create and enter the build directory
		
		Run CMake (simplified, relying on PICO_SDK_PATH being set)
		
		Compile the code (this generates the .uf2 file)

Step 3: Run the micro-ROS Agent (On Linux PC)

You already built the micro-ROS Agent in your previous workspace, so we will use the correct command to launch it:

1. Launch the MicroROS agent...	`source ~/ros_workspaces/microros_ws1/install/setup.bash`
	
	`ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyACM0 -b 115200`
		Source environment
		
		Run the agent, listening on the serial port (Pico defaults to ttyACM0)

2. Flash .uf2 example file onto Pico	In new terminal… `cd pico/src/micro_ros_raspberrypi_pico_sdk/build` ...see column to right for next steps	Now find the “pico_micro_ros_example.uf2” and copy it to Downloads folder Now hold down BOOTSEL button on Pico and plug into PC. Now open the Pico folder on your desktop and drag the “pico_micro_ros_example.uf2” file from Downloads into the Pico folder.

3. Pico reboots automatically… check terminal with ROS agent running for evidence of success	Example logs…
	
	`[1760536650.736963] info  | TermiosAgentLinux.cpp | init | running...  | fd: 30`
	
	`[1760536651.381516] info  | Root.cpp  | create_client | create  | client_key: 0x002799FD, session_id: 0x81`
	
	`[1760536651.381650] info  | SessionManager.hpp | establish_session | session established  | client_key: 0x002799FD, address: 0`
	
	`[1760536651.415037] info  | ProxyClient.cpp  | create_participant | participant created  | client_key: 0x002799FD, participant_id: 0x000(1)`
	
	`[1760536651.419222] info  | ProxyClient.cpp  | create_topic | topic created  | client_key: 0x002799FD, topic_id: 0x000(2), participant_id: 0x000(1)`
	
	`[1760536651.421413] info  | ProxyClient.cpp  | create_publisher | publisher created  | client_key: 0x002799FD, publisher_id: 0x000(3),`	

	participant_id: 0x000(1) [1760536651.424352] info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x002799FD, datawriter_id: 0x000(5), publisher_id: 0x000(3)	
4. Test for full functionality	ros2 node list ros2 topic list ros2 topic echo /pico_publisher	Should see /pico_node Should see /pico_publisher Should see data coming through